

LAPORAN PROYEK AKHIR
PENERAPAN KONSEP OOP PADA APLIKASI PENDATAAN
MAHASISWA BERBASIS GUI MENGGUNAKAN PYTHON
TKINTER

Mata Kuliah:

Pemrograman Berorientasi Objek

Oleh:

Firda Agustina

24091397032

2024B



PROGRAM STUDI MANAJEMEN INFOMATIKA

FAKULTAS VOKASI

UNIVERSITAS NEGERI SURABAYA

2025

DAFTAR ISI

BAB I PENDAHULUAN	3
1.1 Latar Belakang	3
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	4
BAB II LANDASAN TEORI.....	5
2.1 Pemrograman Berorientasi Objek (OOP)	5
2.2 Python	5
2.3 Tkinter	6
BAB III ANALISIS dan PERANCANGAN SISTEM	7
3.1 Analisis Kebutuhan Sistem	7
3.2 Perancangan Class Diagram	8
3.3 Perancangan Antarmuka (GUI)	9
BAB IV IMPLEMENTASI dan PENGUJIAN	10
4.1 Implementasi Program	10
4.2 Kode Program.....	11
4.3 Hasil Pengujian Aplikasi Program	20
4.3.1 Tampilan Utama Aplikasi.....	21
4.3.2 Tambah Data Mahasiswa	21
4.3.3 Edit Data Mahasiswa	22
4.3.4 Hapus Data Mahasiswa.....	23
4.3.5 Pencarian Data.....	24
4.3.6 Export CSV	24
BAB V Penutup.....	26
5.1 Kesimpulan.....	26
5.2 Saran.....	26

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengelolaan data mahasiswa merupakan salah satu kegiatan administrasi yang dilakukan secara berulang dan membutuhkan ketelitian. Data seperti NIM, Nama, Kelas, Program Studi, dan Tahun Masuk harus tersimpan dengan rapi agar mudah diakses kembali. Apabila data tersebut tidak dikelola dengan baik, maka dapat menimbulkan kesulitan dalam proses pencarian maupun pembaruan data, terutama ketika jumlah data sudah semakin banyak.

Pada beberapa kondisi, pengolahan data masih dilakukan secara manual atau hanya disimpan dalam bentuk catatan dan file sederhana. Cara tersebut dinilai kurang efektif karena rawan terjadi kesalahan pencatatan, data ganda, maupun data yang sulit dicari kembali. Selain itu, tampilan data yang kurang terstruktur juga dapat menyulitkan pengguna dalam memahami informasi yang tersedia.

Seiring dengan perkembangan teknologi, pemanfaatan aplikasi komputer menjadi salah satu solusi yang dapat membantu proses pengolahan data. Aplikasi dengan tampilan antarmuka yang sederhana dan mudah digunakan dapat mempermudah pengguna dalam melakukan input, pencarian, pengeditan, serta penghapusan data. Dengan adanya sistem yang terkomputerisasi, proses pengelolaan data dapat dilakukan secara lebih cepat dan efisien.

Berdasarkan permasalahan tersebut, penulis membuat sebuah aplikasi pendataan mahasiswa berbasis Graphical User Interface (GUI) menggunakan bahasa pemrograman Python dengan library Tkinter. Aplikasi ini dirancang untuk menampilkan dan mengelola data mahasiswa secara terstruktur melalui tampilan visual yang mudah dipahami.

Pembuatan aplikasi ini juga bertujuan sebagai bentuk penerapan konsep Pemrograman Berorientasi Objek (PBO) yang telah dipelajari selama perkuliahan. Dengan menerapkan konsep tersebut, diharapkan program yang dibuat memiliki struktur yang jelas, mudah dikembangkan, serta dapat membantu penulis dalam memahami penerapan PBO secara nyata melalui sebuah proyek aplikasi sederhana.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam proyek ini adalah sebagai berikut:

1. Bagaimana merancang sebuah aplikasi pendataan mahasiswa berbasis GUI menggunakan Python dan Tkinter?
2. Bagaimana cara menerapkan konsep Pemrograman Berorientasi Objek (OOP) pada aplikasi pendataan mahasiswa?
3. Bagaimana aplikasi dapat membantu proses pengelolaan data mahasiswa seperti penambahan, pencarian, pengeditan, dan penghapusan data?
4. Bagaimana tampilan antarmuka aplikasi agar mudah digunakan dan dipahami oleh pengguna?

1.3 Tujuan Penelitian

Tujuan dari pembuatan aplikasi pendataan mahasiswa ini adalah untuk merancang dan membangun sebuah aplikasi berbasis Graphical User Interface (GUI) yang dapat digunakan untuk mengelola data mahasiswa secara lebih terstruktur. Aplikasi ini diharapkan mampu membantu pengguna dalam melakukan pengolahan data mahasiswa dengan lebih mudah dan efisien.

Selain itu, tujuan dari proyek ini adalah untuk menerapkan konsep Pemrograman Berorientasi Objek (OOP) yang telah dipelajari selama perkuliahan, seperti penggunaan Class, Object, Inheritance, dan Polymorphism, ke dalam sebuah program nyata. Dengan adanya proyek ini, penulis diharapkan dapat memahami penerapan konsep OOP secara lebih mendalam serta mengetahui bagaimana konsep tersebut digunakan dalam pengembangan aplikasi sederhana.

Tujuan lainnya adalah sebagai pemenuhan tugas Ujian Akhir Semester (UAS) pada mata kuliah Pemrograman Berorientasi Objek, sekaligus melatih kemampuan penulis dalam merancang aplikasi yang memiliki tampilan antarmuka yang mudah digunakan dan fungsionalitas yang sesuai dengan kebutuhan pengguna.

Melalui pembuatan aplikasi ini, penulis juga berharap dapat meningkatkan kemampuan logika pemrograman serta pemahaman dalam mengembangkan program berbasis GUI menggunakan Python.

BAB II

LANDASAN TEORI

2.1 Pemrograman Berorientasi Objek (Object Oriented Programming)

Pemrograman Berorientasi Objek (Object Oriented Programming/OOP) merupakan salah satu paradigma pemrograman yang berfokus pada penggunaan objek sebagai dasar dalam pembuatan program. Setiap objek merepresentasikan suatu entitas yang memiliki data (atribut) dan perilaku (method). Dengan pendekatan ini, program dapat disusun secara lebih terstruktur dan mudah dipahami.

Konsep utama dalam OOP meliputi class dan object, di mana class berfungsi sebagai cetak biru (blueprint) untuk membuat object. Selain itu, OOP juga memiliki konsep lain seperti Encapsulation, Inheritance, dan Polymorphism yang digunakan untuk mengatur akses data, pewarisan sifat, serta fleksibilitas dalam penggunaan method.

Penerapan konsep OOP membuat program lebih terorganisir karena setiap bagian program memiliki tugas dan tanggung jawab masing-masing. Hal ini memudahkan programmer dalam membaca alur program serta mengurangi kemungkinan terjadinya kesalahan pada saat pengembangan.

Pada pembuatan aplikasi pendataan mahasiswa, konsep OOP sangat membantu dalam pengelompokan data mahasiswa ke dalam class tertentu. Dengan adanya pembagian class tersebut, proses pengolahan data menjadi lebih rapi dan sistematis. Selain itu, penggunaan OOP juga memudahkan penulis dalam melakukan pengembangan program apabila di kemudian hari diperlukan penambahan fitur baru.

2.2 Python

Python merupakan salah satu bahasa pemrograman tingkat tinggi yang banyak digunakan karena memiliki sintaks yang sederhana dan mudah dipahami. Bahasa pemrograman ini sangat cocok digunakan oleh pemula, namun tetap cukup kuat untuk membangun aplikasi yang lebih kompleks.

Python mendukung berbagai paradigma pemrograman, salah satunya adalah Pemrograman Berorientasi Objek (OOP). Dengan dukungan tersebut, Python memungkinkan programmer untuk membuat Class, Object, serta menerapkan konsep OOP seperti Encapsulation, Inheritance, dan Polymorphism dengan lebih mudah.

Selain kemudahan sintaks, Python juga memiliki banyak library yang dapat digunakan untuk mendukung pengembangan aplikasi. Library tersebut membantu programmer dalam mempercepat proses pembuatan program tanpa harus menulis kode dari awal.

Pada proyek ini, Python dipilih sebagai bahasa pemrograman utama karena kemudahan dalam penulisan kode dan banyaknya library pendukung yang tersedia. Python juga mendukung pembuatan aplikasi berbasis antarmuka grafis, sehingga sesuai untuk digunakan dalam pengembangan aplikasi pendataan mahasiswa berbasis GUI.

2.3 Tkinter

Tkinter merupakan salah satu library bawaan pada bahasa pemrograman Python yang digunakan untuk membuat aplikasi berbasis Graphical User Interface (GUI). Dengan menggunakan Tkinter, programmer dapat membuat tampilan aplikasi seperti jendela, tombol, label, kolom input, dan komponen lainnya yang memudahkan interaksi antara pengguna dan program.

Penggunaan Tkinter relatif mudah karena tidak memerlukan instalasi tambahan dan sudah terintegrasi langsung dengan Python. Hal ini menjadikan Tkinter cocok digunakan untuk pembuatan aplikasi sederhana hingga menengah, terutama untuk kebutuhan pembelajaran dan proyek perkuliahan.

Tkinter juga memungkinkan pengaturan tata letak komponen antarmuka secara fleksibel, sehingga tampilan aplikasi dapat disesuaikan dengan kebutuhan pengguna. Selain itu, Tkinter mendukung event handling, yaitu kemampuan program untuk merespons tindakan pengguna seperti klik tombol atau pemilihan data.

Pada aplikasi pendataan mahasiswa yang dibuat, Tkinter digunakan untuk menampilkan antarmuka utama aplikasi, daftar data mahasiswa, kolom pencarian, serta tombol-tombol untuk menambah, mengubah, dan menghapus data. Dengan adanya tampilan GUI ini, aplikasi menjadi lebih mudah digunakan dan lebih menarik dibandingkan dengan aplikasi berbasis teks (console).

BAB III

ANALISIS dan PERANCANGAN SISTEM

3.1 Analisis Kebutuhan Sistem

Analisis kebutuhan sistem dilakukan untuk mengetahui kebutuhan yang diperlukan dalam pembuatan aplikasi pendataan mahasiswa. Tahap ini bertujuan agar aplikasi yang dibuat sesuai dengan fungsi yang dibutuhkan pengguna serta dapat berjalan dengan baik dan terstruktur.

Aplikasi pendataan mahasiswa ini dirancang untuk membantu pengguna dalam mengelola data mahasiswa secara sederhana dan terkomputerisasi. Data yang dikelola meliputi NIM, Nama Mahasiswa, Kelas, Program Studi, dan Tahun Masuk. Selain itu, sistem juga membedakan jenis mahasiswa, yaitu mahasiswa reguler dan mahasiswa transfer.

Aplikasi membutuhkan antarmuka berbasis Graphical User Interface (GUI) agar pengguna dapat mengoperasikan sistem dengan mudah tanpa menggunakan perintah berbasis teks. Oleh karena itu, sistem harus menyediakan tampilan daftar data mahasiswa, detail informasi mahasiswa, serta tombol-tombol untuk melakukan pengolahan data.

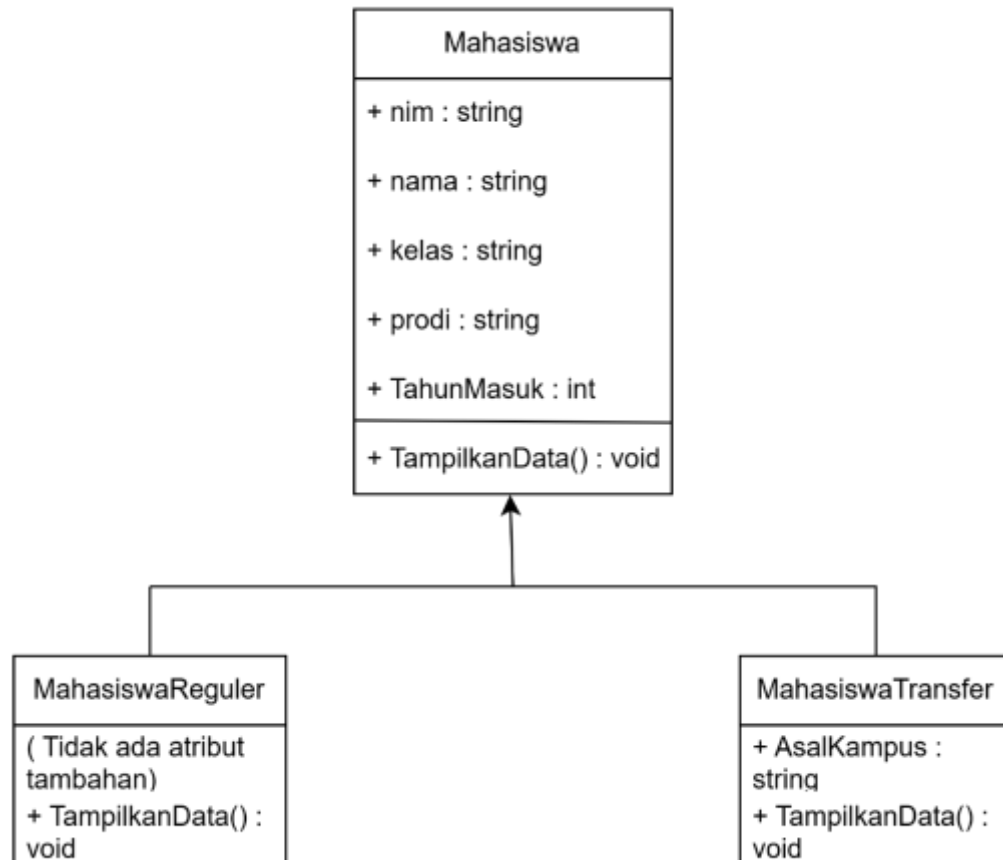
Berdasarkan hasil analisis, kebutuhan fungsional pada aplikasi ini meliputi:

1. Sistem dapat menampilkan data mahasiswa dalam bentuk daftar.
2. Sistem dapat menambahkan data mahasiswa baru.
3. Sistem dapat mengubah data mahasiswa yang sudah tersimpan.
4. Sistem dapat menghapus data mahasiswa.

Sistem dapat melakukan pencarian data berdasarkan NIM, Nama, Program Studi, Kelas, dan Tahun Masuk.

Selain kebutuhan fungsional, aplikasi juga memiliki kebutuhan non-fungsional, yaitu aplikasi harus mudah digunakan (user friendly), memiliki tampilan yang sederhana, serta dapat dijalankan pada perangkat komputer menggunakan bahasa pemrograman Python.

3.2 Perancangan Class Diagram



Class diagram pada aplikasi pendataan mahasiswa digunakan untuk menggambarkan struktur kelas dan hubungan antar kelas yang terdapat pada sistem. Diagram ini terdiri dari satu class induk yaitu Mahasiswa dan dua class turunan yaitu MahasiswaReguler dan MahasiswaTransfer.

Class Mahasiswa menyimpan atribut umum yang dimiliki oleh seluruh mahasiswa, seperti NIM, Nama, Kelas, Program Studi, dan Tahun Masuk. Class MahasiswaReguler dan MahasiswaTransfer mewarisi atribut dan method dari class Mahasiswa. Perbedaan terdapat pada class MahasiswaTransfer yang memiliki atribut tambahan berupa asal kampus.

Hubungan antar class pada diagram menunjukkan penerapan konsep Inheritance, di mana class MahasiswaReguler dan MahasiswaTransfer merupakan turunan dari class Mahasiswa. Selain itu, penggunaan method yang sama pada class turunan menunjukkan penerapan konsep polymorphism dalam sistem.

3.3 Perancangan Antarmuka (Graphical User Interface)

Perancangan antarmuka dilakukan untuk merancang tampilan aplikasi pendataan mahasiswa sebelum diimplementasikan ke dalam bentuk program. Perancangan ini bertujuan agar aplikasi memiliki tampilan yang sederhana, rapi, dan mudah digunakan oleh pengguna.

Antarmuka aplikasi dirancang berbasis Graphical User Interface (GUI) menggunakan library Tkinter. Pada rancangan antarmuka terdapat beberapa komponen utama, seperti form input data mahasiswa yang berisi NIM, Nama, Kelas, Program Studi, Tahun Masuk, dan Asal Kampus (untuk mahasiswa transfer). Selain itu, disediakan juga tabel atau daftar data mahasiswa untuk menampilkan seluruh data yang telah dimasukkan.

Aplikasi juga dirancang memiliki tombol-tombol fungsi, seperti tombol tambah data, ubah data, hapus data, dan cari data mahasiswa. Dengan adanya perancangan antarmuka ini, diharapkan proses implementasi aplikasi dapat berjalan lebih terstruktur dan sesuai dengan kebutuhan pengguna.

Selain itu, perancangan antarmuka juga memperhatikan kemudahan navigasi antar komponen agar pengguna tidak mengalami kesulitan saat menggunakan aplikasi. Tata letak komponen disusun secara sederhana untuk menghindari tampilan yang terlalu padat.

BAB IV

IMPLEMENTASI dan PENGUJIAN SISTEM

4.1 Implementasi Program

Tahap implementasi program merupakan tahap penerapan dari hasil perancangan sistem yang telah dibuat pada bab sebelumnya. Pada tahap ini, rancangan class diagram dan perancangan antarmuka yang telah dirancang kemudian diimplementasikan ke dalam bentuk program.

Aplikasi pendataan mahasiswa ini dikembangkan menggunakan bahasa pemrograman Python dengan bantuan library Tkinter untuk membangun antarmuka berbasis Graphical User Interface (GUI). Pemilihan Python dan Tkinter didasarkan pada kemudahan dalam penulisan kode serta kemampuannya dalam membangun aplikasi desktop sederhana.

Dalam proses implementasi, konsep Pemrograman Berorientasi Objek (OOP) diterapkan melalui penggunaan class dan object yang sesuai dengan perancangan sistem. Penerapan konsep OOP bertujuan agar struktur program lebih terorganisir, mudah dikembangkan, serta mudah dipahami.

Implementasi program dilakukan agar aplikasi mampu mengelola data mahasiswa secara terstruktur, mulai dari proses input data mahasiswa, menampilkan data mahasiswa, mengubah data, menghapus data, serta melakukan pencarian data berdasarkan kriteria tertentu. Dengan adanya tahap implementasi ini, rancangan sistem yang telah dibuat dapat diwujudkan menjadi aplikasi yang dapat dijalankan dan digunakan oleh pengguna.

4.2 Kode Program

```
import tkinter as tk
from tkinter import messagebox, ttk
import csv
```

Kode ini digunakan untuk memanggil library yang dibutuhkan, yaitu Tkinter untuk membuat tampilan GUI, messagebox untuk menampilkan pesan, dan csv untuk menyimpan data mahasiswa ke file CSV.

```
class Mahasiswa:
    def __init__(self, nim, nama, kls, prodi, tahun_masuk):
        self._nim = nim
        self._nama = nama
        self._cls = kls
        self._prodi = prodi
```

```
self._tahun_masuk = tahun_masuk
```

Kode ini membuat class Mahasiswa sebagai class induk untuk menyimpan data utama mahasiswa. Method `__init__` digunakan untuk menginisialisasi data awal seperti NIM, Nama, Kelas, Program Studi, dan Tahun Masuk.

```
# Getter
def get_nim(self): return self._nim
def get_nama(self): return self._nama
def get_kls(self): return self._kls
def get_prodi(self): return self._prodi
def get_tahun_masuk(self): return self._tahun_masuk
```

Bagian ini merupakan method getter yang digunakan untuk mengambil nilai dari masing-masing atribut mahasiswa.

```
# Setter
def set_nama(self, val): self._nama = val
def set_kls(self, val): self._kls = val
def set_prodi(self, val): self._prodi = val
def set_tahun_masuk(self, val): self._tahun_masuk = val
```

Method setter digunakan untuk mengubah data mahasiswa tanpa mengakses atribut secara langsung.

```
def display_info(self):
    return (f"NIM: {self._nim}\n"
            f>Nama: {self._nama}\n"
            f"Kelas: {self._kls}\n"
            f"Program Studi: {self._prodi}\n"
            f"Tahun Masuk: {self._tahun_masuk}\n")
```

Method ini digunakan untuk menampilkan data mahasiswa dalam bentuk teks agar mudah ditampilkan pada GUI.

```
class MahasiswaReguler(Mahasiswa):
    def display_info(self):
        return "[REGULER]\n" + super().display_info()
```

Class MahasiswaReguler merupakan turunan dari class Mahasiswa. Method `display_info()` dioverride untuk menambahkan keterangan jenis mahasiswa reguler.

```
class MahasiswaTransfer(Mahasiswa):
    def __init__(self, nim, nama, kls, prodi, tahun_masuk, asal_kampus):
        super().__init__(nim, nama, kls, prodi, tahun_masuk)
        self._asal_kampus = asal_kampus
```

Class MahasiswaTransfer merupakan turunan dari Mahasiswa yang memiliki atribut tambahan berupa asal kampus.

```
def get_asal_kampus(self): return self._asal_kampus
def set_asal_kampus(self, val): self._asal_kampus = val
```

```
def display_info(self):
    return (f"[TRANSFER]\n"
            f"{super().display_info()}"
            f"Asal Kampus: {self._asal_kampus}\n")
```

Method ini digunakan untuk mengelola dan menampilkan data mahasiswa transfer.

```
class Database:
    def __init__(self):
        self._data = []
```

Class Database digunakan sebagai tempat penyimpanan data mahasiswa selama aplikasi berjalan. Data disimpan dalam sebuah list agar mudah dikelola, seperti ditambah, diubah, atau dihapus.

```
def add(self, mhs):
    self._data.append(mhs)

    def update(self, index, new_mhs):
        self._data[index] = new_mhs

    def delete(self, index):
        del self._data[index]
```

Method add() digunakan untuk menambahkan data mahasiswa ke dalam database. Method update() digunakan untuk mengubah data mahasiswa berdasarkan indeks tertentu, sedangkan method delete() digunakan untuk menghapus data mahasiswa yang dipilih.

```
def all(self):
    return self._data

# ✨ Pencarian Lengkap (NIM / NAMA / PRODI / KELAS / TAHUN MASUK)
def search(self, keyword):
    q = keyword.lower()
    return [
        m for m in self._data
        if q in m.get_nim().lower()
        or q in m.get_nama().lower()
        or q in m.get_prodi().lower()
        or q in m.get_kls().lower()
        or q in str(m.get_tahun_masuk())
    ]
```

Method all() digunakan untuk mengambil seluruh data mahasiswa. Method search() digunakan untuk mencari data mahasiswa berdasarkan kata kunci yang dicocokkan dengan NIM, nama, program studi, kelas, atau tahun masuk, sehingga pencarian menjadi lebih fleksibel.

```
class App:
    def __init__(self, root):
        self.db = Database()
        self.root = root
        self.root.title("Data Mahasiswa - UAS PBO (Final)")
        self.root.geometry("1050x550")
        self.root.resizable(False, False)
```

Class App merupakan class utama yang mengatur jalannya aplikasi. Pada bagian ini dibuat objek database dan diatur properti jendela aplikasi seperti judul, ukuran, dan pengaturan agar ukuran jendela tidak dapat diubah.

```
        self.setup_fonts()
        self.build_gui()
        self.seed_demo()
        self.refresh_list()
```

Kode ini digunakan untuk memanggil fungsi-fungsi awal yang dibutuhkan saat aplikasi dijalankan, yaitu pengaturan font, pembuatan tampilan GUI, pengisian data awal, dan menampilkan data mahasiswa ke dalam daftar.

```
def setup_fonts(self):
    self.font_title = ("Segoe UI", 12, "bold")
    self.font_normal = ("Segoe UI", 10)
```

Fungsi ini digunakan untuk mengatur jenis dan ukuran font yang digunakan pada tampilan aplikasi agar terlihat rapi dan mudah dibaca.

```
def build_gui(self):
    top = tk.Frame(self.root, bg="#f2f2f2")
    top.pack(fill="x")

    tk.Label(top, text="Data Mahasiswa", bg="#f2f2f2", font=self.font_title) \
        .pack(side="left", padx=10, pady=10)

    tk.Label(top, text="Cari (NIM / Nama / Prodi / Kelas / Tahun Masuk):",
bg="#f2f2f2") \
        .pack(side="left", padx=5)

    self.entry_cari = tk.Entry(top, width=40)
    self.entry_cari.pack(side="left", padx=5)

    tk.Button(top, text="Cari", width=12, command=self.cari) \
        .pack(side="left", padx=5)

    tk.Button(top, text="Tampilkan Semua", width=15,
command=self.refresh_list) \
```

```
.pack(side="left", padx=5)
```

Bagian ini digunakan untuk membuat header aplikasi dan fitur pencarian data mahasiswa. Terdapat judul aplikasi, kolom input pencarian, serta tombol untuk melakukan pencarian dan menampilkan kembali seluruh data mahasiswa.

```
body = tk.Frame(self.root)
body.pack(fill="both", expand=True, padx=10)
```

Kode ini digunakan untuk membuat area utama aplikasi yang berfungsi sebagai tempat komponen lain, seperti daftar mahasiswa dan detail data.

```
# LISTBOX
left_frame = tk.Frame(body)
left_frame.pack(side="left", fill="y")

tk.Label(left_frame, text="Daftar Mahasiswa", font=self.font_title) \
    .pack(pady=5)

self.listbox = tk.Listbox(left_frame, width=40, height=20,
font=self.font_normal)
self.listbox.pack(side="left", fill="y")
self.listbox.bind("<<ListboxSelect>>", self.show_detail)
```

Bagian ini digunakan untuk menampilkan daftar mahasiswa dalam bentuk listbox. Data mahasiswa ditampilkan secara ringkas, dan saat salah satu data dipilih, sistem akan memanggil fungsi untuk menampilkan detail mahasiswa tersebut.

```
scrollbar = tk.Scrollbar(left_frame, orient="vertical",
command=self.listbox.yview)
scrollbar.pack(side="right", fill="y")
self.listbox.config(yscrollcommand=scrollbar.set)
```

Scrollbar ditambahkan agar pengguna dapat melihat seluruh data mahasiswa jika jumlah data cukup banyak dan tidak muat ditampilkan dalam satu layar.

```
# DETAIL BOX
right_frame = tk.Frame(body)
right_frame.pack(side="left", fill="both", expand=True, padx=10)

tk.Label(right_frame, text="Detail Mahasiswa", font=self.font_title) \
    .pack(pady=5)

self.text_detail = tk.Text(right_frame, height=15, font=self.font_normal)
self.text_detail.pack(fill="both", expand=True)
```

Bagian ini digunakan untuk menampilkan detail lengkap mahasiswa yang dipilih dari daftar. Informasi ditampilkan dalam bentuk teks agar mudah dibaca.

```
# BUTTONS
btn_frame = tk.Frame(right_frame)
```

```

        btn_frame.pack(pady=10)

        tk.Button(btn_frame, text="Tambah Reguler", width=15,
command=self.tambah_reguler).grid(row=0, column=0, padx=5)
        tk.Button(btn_frame, text="Tambah Transfer", width=15,
command=self.tambah_transfer).grid(row=0, column=1, padx=5)
        tk.Button(btn_frame, text="Edit Terpilih", width=15,
command=self.edit_data).grid(row=0, column=2, padx=5)
        tk.Button(btn_frame, text="Hapus Terpilih", width=15,
command=self.hapus_data).grid(row=0, column=3, padx=5)
        tk.Button(btn_frame, text="Export CSV", width=15,
command=self.export_csv).grid(row=0, column=4, padx=5)

```

Kode ini digunakan untuk membuat tombol-tombol utama dalam aplikasi, yaitu menambah data mahasiswa reguler dan transfer, mengedit data yang dipilih, menghapus data, serta mengekspor data ke dalam file CSV.

```

# STATUS BAR
        self.status = tk.Label(self.root, text="Jumlah Mahasiswa: 0", anchor="w")
        self.status.pack(fill="x")

```

Status bar berfungsi untuk menampilkan informasi jumlah total mahasiswa yang tersimpan di dalam aplikasi.

```

def seed_demo(self):
    data = [
        MahasiswaReguler("24100101", "Andi Pratama", "2024B", "Sistem
Informasi", 2024),
        MahasiswaTransfer("24100102", "Erika Putri", "2024C", "Manajemen
Informatika", 2024, "WIJAYA KUSUMA"),
        MahasiswaReguler("23100201", "Bunga Lestari", "2023B", "Manajemen
Informatika", 2023),
        MahasiswaTransfer("23100202", "Farel Nugroho", "2023C", "Teknik
Komputer", 2023, "UPN"),
        MahasiswaReguler("22100301", "Cici Ramadhani", "2022A", "Teknik
Komputer", 2022),
        MahasiswaReguler("22100302", "Gina Marlina", "2022C", "Sistem
Informasi", 2022),
        MahasiswaTransfer("21100401", "Dimas Setiawan", "2021B", "Sistem
Informasi", 2021, "UINSA"),
        MahasiswaTransfer("21100402", "Hendra Wijaya", "2021C", "Manajemen
Informatika", 2021, "UT"),
    ]

    for d in data:
        self.db.add(d)

```

Bagian ini digunakan untuk mengisi data awal mahasiswa sebagai contoh saat aplikasi pertama kali dijalankan, sehingga tampilan tidak kosong dan dapat langsung diuji.

```
def refresh_list(self):
    self.listbox.delete(0, tk.END)
    for i, m in enumerate(self.db.all(), start=1):
        self.listbox.insert(tk.END, f"[{i:02}]
{m.get_nim()} {m.get_nama()}")
    self.status.config(text=f"Jumlah Mahasiswa: {len(self.db.all())}")
```

Kode ini berfungsi untuk memperbarui daftar mahasiswa di listbox setiap kali terjadi perubahan data, seperti penambahan, pengeditan, atau penghapusan data.

```
def show_detail(self, evt):
    index = self.get_selected_index()
    if index is None:
        return

    m = self.db.all()[index]
    self.text_detail.delete("1.0", tk.END)
    self.text_detail.insert(tk.END, m.display_info())
```

Kode ini berfungsi untuk menampilkan detail lengkap mahasiswa ketika salah satu data dipilih pada listbox. Informasi diambil dari database lalu ditampilkan pada kotak teks.

```
def get_selected_index(self):
    try:
        return self.listbox.curselection()[0]
    except:
        return None
```

Fungsi ini digunakan untuk mengetahui data mahasiswa yang sedang dipilih. Jika tidak ada data yang dipilih, maka fungsi akan mengembalikan nilai kosong.

```
def cari(self):
    keyword = self.entry_cari.get().strip()
    hasil = self.db.search(keyword)

    self.listbox.delete(0, tk.END)

    for i, m in enumerate(hasil, start=1):
        self.listbox.insert(tk.END, f"[{i:02}]
{m.get_nim()} {m.get_nama()}")
```

Bagian ini digunakan untuk melakukan pencarian data mahasiswa berdasarkan kata kunci yang dimasukkan, seperti NIM, Nama, Program Studi, Kelas, atau Tahun Masuk.

```
def tambah_reguler(self):
    self.popup_form("reguler")
```



```
def tambah_transfer(self):
    self.popup_form("transfer")
```

Kedua method ini digunakan untuk menambah data mahasiswa baru. Masing-masing memanggil popup_form() dengan tipe mahasiswa yang sesuai, sehingga form yang muncul menyesuaikan kebutuhan data reguler atau transfer.

```
def edit_data(self):
    index = self.get_selected_index()
    if index is None:
        messagebox.showwarning("Peringatan", "Pilih data terlebih dahulu.")
        return

    m = self.db.all()[index]
    jenis = "transfer" if isinstance(m, MahasiswaTransfer) else "reguler"
    self.popup_form(jenis, index, m)
```

Method ini digunakan untuk mengedit data mahasiswa yang dipilih. Program akan memeriksa jenis mahasiswa (reguler atau transfer) dan memanggil form yang sesuai dengan data lama sebagai referensi pengisian.

```
def hapus_data(self):
    index = self.get_selected_index()
    if index is None:
        messagebox.showwarning("Peringatan", "Tidak ada data yang dipilih!")
        return

    if messagebox.askyesno("Konfirmasi", "Hapus data terpilih?"):
        self.db.delete(index)
        self.refresh_list()
        self.text_detail.delete("1.0", tk.END)
```

Bagian ini memungkinkan pengguna untuk menghapus data mahasiswa yang dipilih. Program akan menampilkan konfirmasi sebelum menghapus, lalu memperbarui daftar mahasiswa dan mengosongkan tampilan detail.

```
def popup_form(self, jenis, index=None, mhs=None):
    win = tk.Toplevel(self.root)
    win.title("Form Mahasiswa")
    win.geometry("400x400")
```

Kode ini membuat jendela baru menggunakan Toplevel yang terpisah dari jendela utama aplikasi. Jendela ini diberi judul Form Mahasiswa dan ukuran tertentu agar tampilan form lebih rapi.

```
labels = ["NIM", "Nama", "Kelas", "Program Studi", "Tahun Masuk"]
entries = {}
```

Bagian ini mendefinisikan field data mahasiswa yang akan diinput. Dictionary entries digunakan untuk menyimpan setiap input agar mudah diakses saat proses penyimpanan data.

```
for i, lbl in enumerate(labels):
    tk.Label(win, text=lbl).pack()
    ent = tk.Entry(win, width=30)
    ent.pack()
    entries[lbl] = ent
```

Kode ini membuat label dan kotak input secara otomatis menggunakan perulangan. Cara ini membuat kode lebih efisien dan tidak perlu menulis field satu per satu.

```
# Field tambahan jika Transfer
tk.Label(win, text="Asal Kampus (Transfer)").pack()
entry_asal = tk.Entry(win, width=30)
entry_asal.pack()
```

Field Asal Kampus disediakan sebagai data tambahan khusus untuk mahasiswa transfer. Field ini akan digunakan saat jenis mahasiswa adalah transfer.

```
if mhs:
```

Jika parameter mhs berisi data, berarti form dibuka dalam kondisi edit data. Data lama akan ditampilkan kembali ke dalam form sebagai referensi pengisian.

```
entries["NIM"].insert(0, mhs.get_nim())
entries["Nama"].insert(0, mhs.get_nama())
entries["Kelas"].insert(0, mhs.get_kls())
entries["Program Studi"].insert(0, mhs.get_prodi())
entries["Tahun Masuk"].insert(0, mhs.get_tahun_masuk())
```

Setiap atribut mahasiswa lama diambil melalui method getter dan dimasukkan ke field yang sesuai.

```
if jenis == "transfer":
    entry_asal.insert(0, mhs.get_asal_kampus())
```

Jika mahasiswa bertipe transfer, maka field asal kampus juga akan diisi dengan data sebelumnya.

```
def simpan():
```

Fungsi ini dijalankan ketika tombol Simpan ditekan dan berfungsi untuk memproses penyimpanan data mahasiswa.

```
nim = entries["NIM"].get().strip()
nama = entries["Nama"].get().strip()
kls = entries["Kelas"].get().strip()
prodi = entries["Program Studi"].get().strip()
tahun = entries["Tahun Masuk"].get().strip()
```

Data diambil dari setiap field input dan dibersihkan dari spasi berlebih agar data yang disimpan lebih rapi.

```
if jenis == "reguler":
    new = MahasiswaReguler(nim, nama, kls, prodi, int(tahun))
else:
    asal = entry_asal.get().strip()
    new = MahasiswaTransfer(nim, nama, kls, prodi, int(tahun), asal)
```

Bagian ini membuat objek mahasiswa sesuai dengan jenisnya. Jika reguler, maka dibuat objek MahasiswaReguler, sedangkan jika transfer dibuat objek MahasiswaTransfer.

```
if mhs is None:
    self.db.add(new)
else:
    self.db.update(index, new)
```

Jika data belum ada sebelumnya, maka data baru akan ditambahkan. Jika data sudah ada, maka data lama akan diperbarui berdasarkan index yang dipilih.

```
self.refresh_list()
win.destroy()
```

Setelah data disimpan, daftar mahasiswa akan diperbarui dan jendela popup akan ditutup.

```
tk.Button(win, text="Simpan", width=15, command=simpan).pack(pady=10)
```

Tombol ini digunakan untuk menjalankan fungsi simpan() ketika ditekan oleh pengguna.

```
with open("data_mahasiswa.csv", "w", newline="") as f:
```

File CSV dibuat atau ditimpa jika sebelumnya sudah ada.

```
w.writerow(["NIM", "Nama", "Kelas", "Prodi", "Tahun Masuk", "Jenis",
"Asal Kampus"])
```

Header ditulis untuk memberikan keterangan setiap kolom data.

```
for m in self.db.all():
```

Program membaca seluruh data mahasiswa yang tersimpan di database.

```
jenis = "Transfer" if isinstance(m, MahasiswaTransfer) else "Reguler"
asal = m.get_asal_kampus() if jenis == "Transfer" else "-"
```

Jenis mahasiswa ditentukan berdasarkan tipe objek, sedangkan asal kampus hanya diisi jika mahasiswa transfer.

```
w.writerow([m.get_nim(), m.get_nama(), m.get_kls(),
m.get_prodi(), m.get_tahun_masuk(), jenis, asal])
```

Setiap data mahasiswa disimpan dalam satu baris file CSV.

```
messagebox.showinfo("Berhasil", "Data berhasil diexport ke CSV!")
```

Pesan ini menandakan bahwa proses export data telah berhasil dilakukan.

```
if __name__ == "__main__":
```

```
root = tk.Tk()
App(root)
root.mainloop()
```

Bagian ini merupakan titik awal program yang menjalankan aplikasi dan menjaga agar aplikasi tetap berjalan.

ANALISIS:

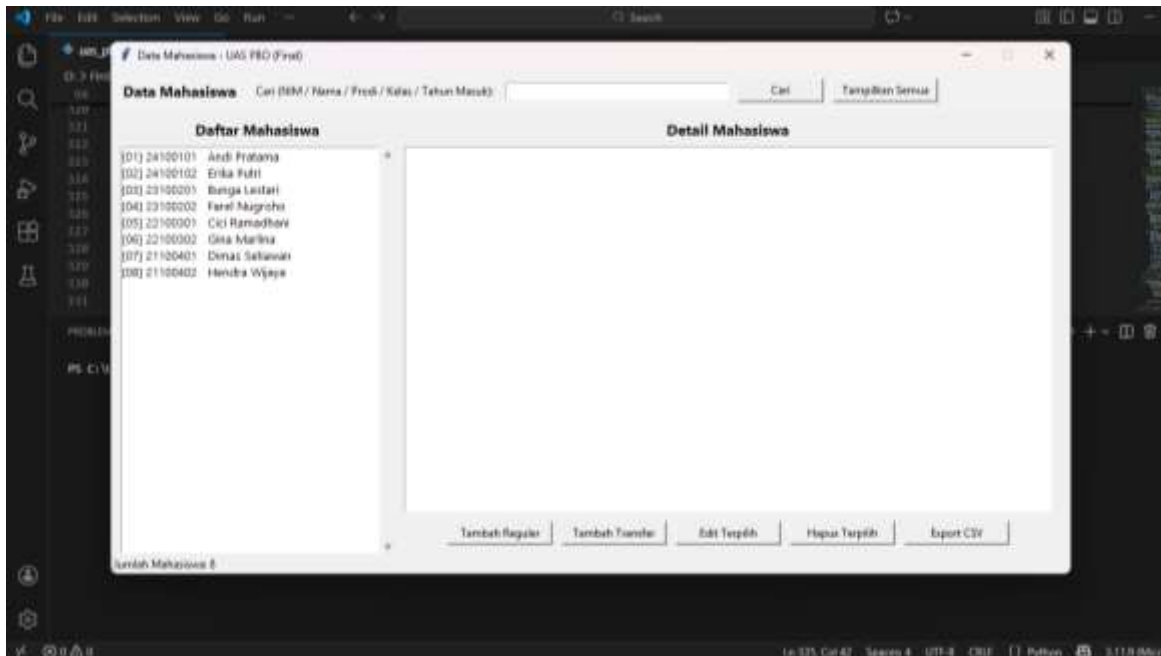
Secara keseluruhan, kode program ini merupakan aplikasi pengelolaan data mahasiswa berbasis GUI menggunakan Python dan Tkinter yang menerapkan konsep Pemrograman Berorientasi Objek. Program memungkinkan pengguna untuk menambah, mengedit, menghapus, dan menampilkan data mahasiswa baik reguler maupun transfer melalui form popup yang dinamis sesuai jenis mahasiswa. Setiap data disimpan dalam objek yang berbeda untuk menunjukkan penerapan Inheritance, sementara proses pengolahan data dikelola melalui kelas database agar lebih terstruktur. Selain itu, program juga menyediakan fitur export data ke file CSV sehingga data dapat digunakan kembali di luar aplikasi. Dengan alur program yang jelas, penggunaan method yang terpisah sesuai fungsinya, serta tampilan yang interaktif, aplikasi ini sudah memenuhi kebutuhan dasar pengelolaan data mahasiswa secara sederhana namun efektif.

4.3 Hasil Pengujian Aplikasi Program

Pengujian aplikasi ini dilakukan untuk memastikan bahwa aplikasi pendataan mahasiswa dapat berjalan dengan baik serta sesuai dengan fungsi yang telah dirancang pada tahap sebelumnya. Pengujian dilakukan dengan menjalankan aplikasi secara langsung dan mencoba seluruh fitur utama melalui antarmuka Graphical User Interface (GUI), mulai dari menampilkan data mahasiswa hingga melakukan pengolahan data.

Hasil pengujian ditunjukkan melalui beberapa screenshot tampilan aplikasi yang mewakili setiap fitur yang diuji, seperti penambahan data, pengeditan, penghapusan, pencarian, dan ekspor data. Setiap screenshot disertai dengan penjelasan singkat sebagai bukti bahwa fitur-fitur tersebut telah berjalan dengan baik dan aplikasi siap digunakan sesuai dengan kebutuhan pengguna.

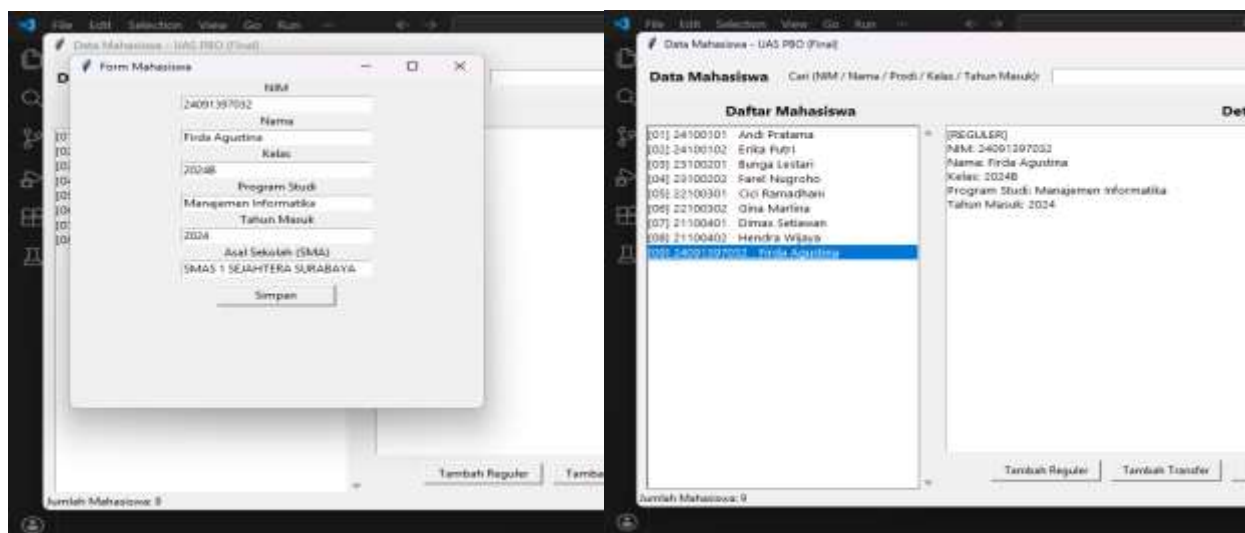
4.3.1 Tampilan Utama Aplikasi



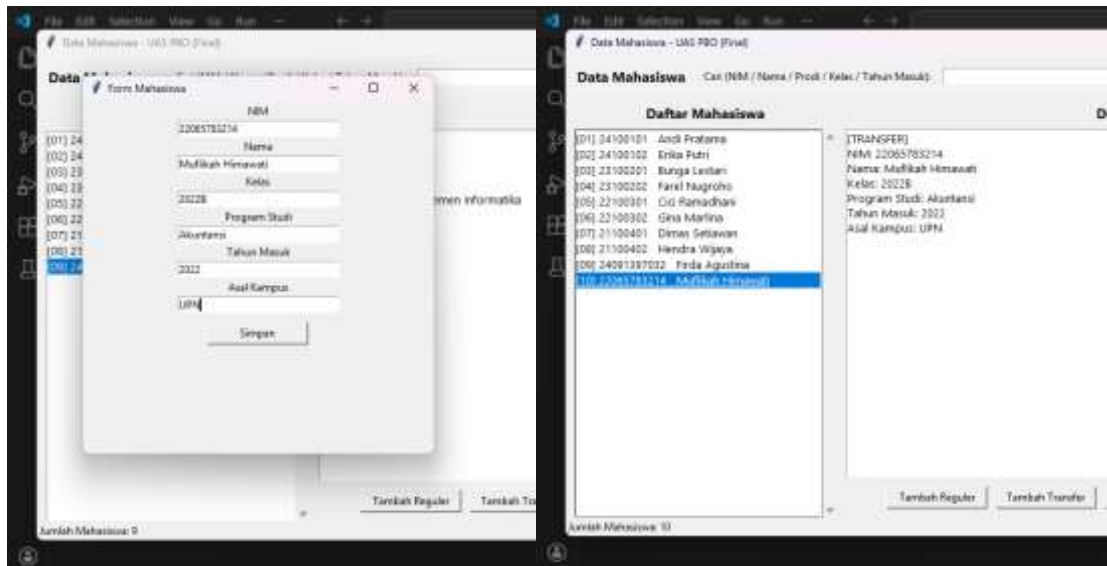
Gambar berikut menunjukkan tampilan utama aplikasi pendataan mahasiswa setelah program dijalankan. Pada tampilan ini ditampilkan daftar mahasiswa, kolom pencarian, serta tombol-tombol fungsi seperti tambah data, edit data, hapus data, dan export CSV. Tampilan utama ini menunjukkan bahwa aplikasi dapat berjalan dengan baik dan siap digunakan untuk pengolahan data mahasiswa.

4.3.2 Mengujian Penambahan Data Mahasiswa

- Mahasiswa Reguler



- Mahasiswa Transfer



Menambah Mahasiswa Reguler: Gambar tersebut menunjukkan pengujian fitur penambahan data mahasiswa reguler. Pada form ini, pengguna mengisi data mahasiswa seperti NIM, nama, kelas, program studi, tahun masuk, serta asal sekolah (SMA). Setelah tombol simpan ditekan, data mahasiswa reguler berhasil disimpan dan langsung ditampilkan pada daftar mahasiswa, yang menandakan bahwa fitur penambahan mahasiswa reguler berjalan dengan baik.

Menambah Mahasiswa Transfer: Gambar tersebut menunjukkan pengujian fitur penambahan data mahasiswa transfer. Pada form ini, pengguna mengisi data mahasiswa seperti NIM, nama, kelas, program studi, tahun masuk, serta asal kampus sebelumnya. Setelah data disimpan, informasi mahasiswa transfer berhasil ditambahkan ke dalam daftar mahasiswa dan dapat ditampilkan dengan benar, sehingga fitur penambahan mahasiswa transfer dapat berjalan sesuai dengan yang dirancang.

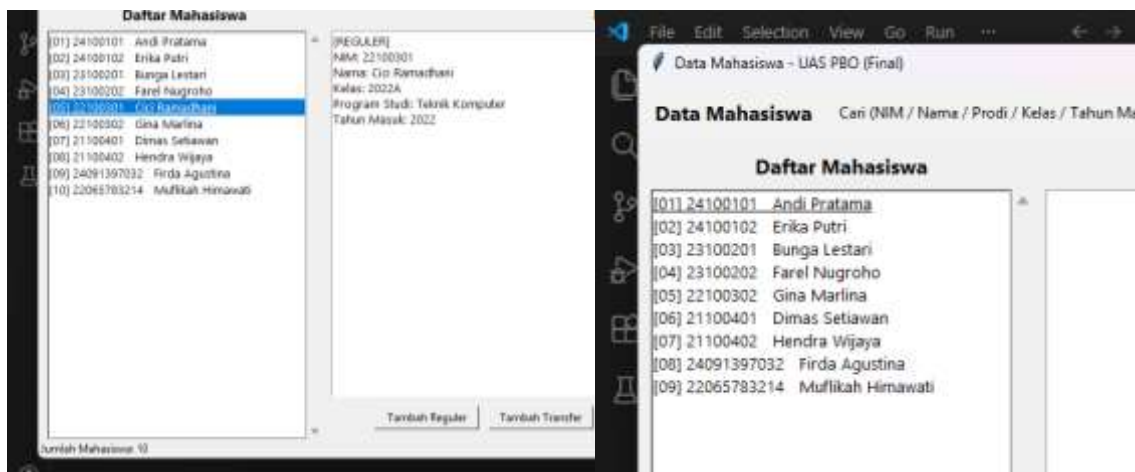
4.3.3 Edit Data Mahasiswa



Gambar berikut menunjukkan pengujian fitur edit data mahasiswa. Pada proses ini, pengguna memilih salah satu data mahasiswa yang terdapat pada daftar mahasiswa, kemudian menekan tombol Edit Terpilih. Sistem akan menampilkan form edit yang berisi data lama sebagai referensi pengisian.

Setelah pengguna melakukan perubahan data dan menekan tombol simpan, data mahasiswa berhasil diperbarui dan perubahan tersebut langsung ditampilkan pada daftar mahasiswa. Hal ini menunjukkan bahwa fitur edit data mahasiswa dapat berjalan dengan baik dan sesuai dengan fungsi yang dirancang.

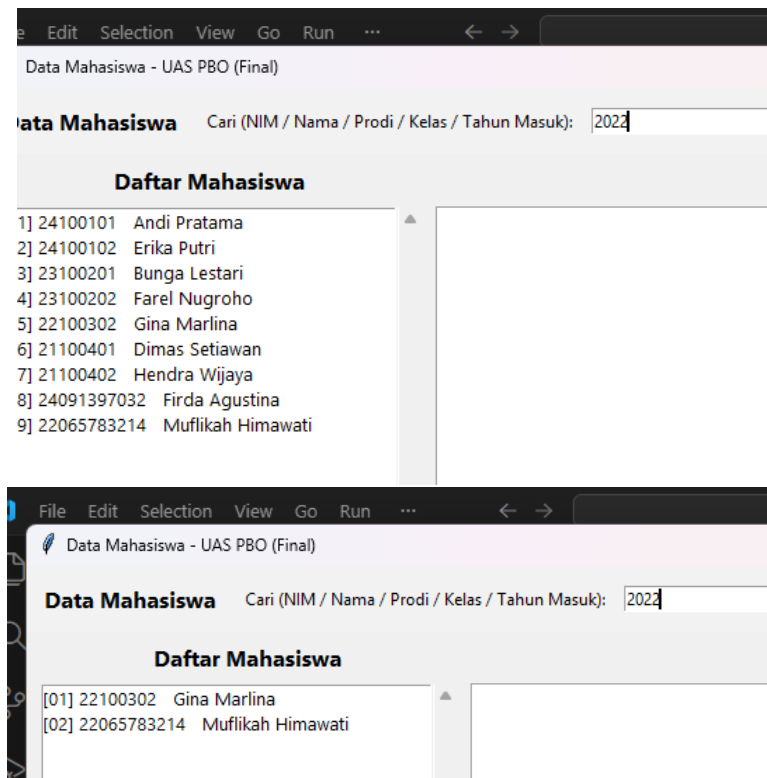
4.3.4 Hapus Data Mahasiswa



Gambar berikut menunjukkan pengujian fitur hapus data mahasiswa. Pengguna memilih salah satu data mahasiswa pada daftar mahasiswa, kemudian menekan tombol Hapus Terpilih. Sistem akan menampilkan pesan konfirmasi sebelum data dihapus.

Setelah pengguna menyetujui proses penghapusan, data mahasiswa yang dipilih berhasil dihapus dari daftar mahasiswa. Hal ini menunjukkan bahwa fitur hapus data mahasiswa dapat berjalan dengan baik dan sesuai dengan fungsi yang telah dirancang.

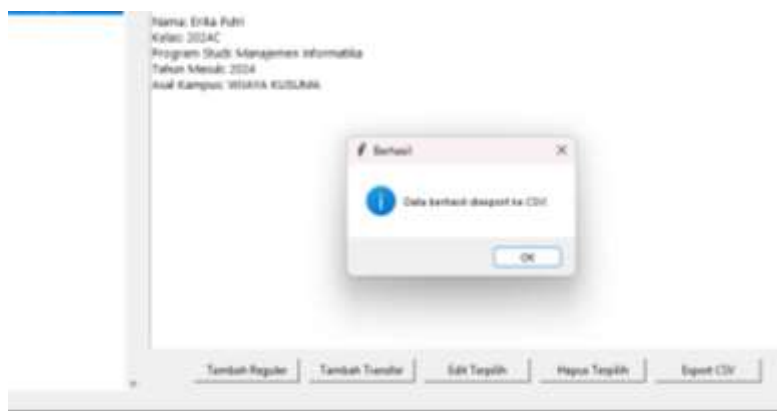
4.3.5 Pencarian Data



Gambar berikut menunjukkan pengujian fitur pencarian data mahasiswa. Pada fitur ini, pengguna dapat melakukan pencarian dengan memasukkan kata kunci berupa NIM, Nama, Kelas, Program Studi, atau Tahun Masuk pada kolom pencarian.

Hasil pencarian akan menampilkan data mahasiswa yang sesuai dengan kata kunci yang dimasukkan. Hal ini menunjukkan bahwa fitur pencarian data mahasiswa dapat berjalan dengan baik dan membantu pengguna dalam menemukan data secara lebih cepat.

4.3.6 Pengujian Export CSV



	A	B	C	D	E	F	G
1	NIM	Nama	Kelas	Program Studi	Tahun Masuk	Jenis	Asal Kampus / Sekolah
2	24100101	Andi Pratama	2024B	Sistem Informasi		2024 Reguler	SMAS 1 Sejahtera Surabaya
3	24100102	Erika Putri	2024C	Manajemen Informatika		2024 Transfer	WIJAYA KUSUMA
4	23100201	Bunga Lestari	2023B	Manajemen Informatika		2023 Reguler	SMAS 1 Sejahtera Surabaya
5	23100202	Farel Nugroho	2023C	Teknik Komputer		2023 Transfer	UPN
6	22100302	Gina Marlina	2022C	Sistem Informasi		2022 Reguler	SMAS 1 Sejahtera Surabaya
7	21100401	Dimas Setiawan	2021B	Sistem Informasi		2021 Transfer	UINSA
8	21100402	Hendra Wijaya	2021C	Manajemen Informatika		2021 Transfer	UT
9	24091397032	Firda Agustina	2024B	Manajemen Informatika		2024 Reguler	SMAS 1 Sejahtera Surabaya
10	22065783214	Muflikah Himawati	2022B	Akuntansi		2022 Transfer	UPN
11							

Gambar berikut menunjukkan pengujian fitur export data mahasiswa ke dalam format CSV. Pada proses ini, pengguna menekan tombol Export CSV untuk menyimpan seluruh data mahasiswa yang ada pada aplikasi.

Setelah proses export dilakukan, data mahasiswa berhasil disimpan ke dalam file CSV dan dapat dibuka menggunakan aplikasi spreadsheet seperti Microsoft Excel. Hal ini menunjukkan bahwa fitur export data ke CSV dapat berjalan dengan baik sesuai dengan fungsi yang dirancang.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa aplikasi pendataan mahasiswa berbasis Python dan Tkinter ini berhasil dibuat dan dapat berjalan dengan baik sesuai dengan tujuan awal. Aplikasi ini mampu mengelola data mahasiswa reguler dan mahasiswa transfer dengan fitur utama seperti penambahan data, pengeditan data, penghapusan data, pencarian data, serta export data ke file CSV. Seluruh fitur tersebut telah diuji melalui antarmuka Graphical User Interface (GUI) dan menunjukkan hasil yang sesuai dengan perancangan sistem.

Selain itu, penerapan konsep Pemrograman Berorientasi Objek (PBO) pada aplikasi ini telah diterapkan dengan baik melalui penggunaan class, pewarisan (inheritance), dan enkapsulasi. Struktur program yang digunakan membuat kode lebih terorganisir, mudah dipahami, dan mudah dikembangkan di kemudian hari. Dengan adanya aplikasi ini, proses pendataan mahasiswa menjadi lebih efektif dan efisien dibandingkan dengan proses pencatatan manual.

5.2 Saran

Aplikasi pendataan mahasiswa ini masih memiliki peluang untuk dikembangkan lebih lanjut agar menjadi lebih optimal. Beberapa pengembangan yang dapat dilakukan antara lain menambahkan fitur validasi data input untuk mencegah kesalahan pengisian, serta menambahkan penyimpanan data berbasis database agar data dapat tersimpan secara permanen meskipun aplikasi ditutup.

Selain dari sisi fungsional, pengembangan juga dapat dilakukan pada sisi tampilan antarmuka agar lebih menarik dan user friendly, misalnya dengan menambahkan tabel data, pengurutan data, atau filter berdasarkan jenis mahasiswa. Dengan adanya pengembangan tersebut, diharapkan aplikasi ini dapat memberikan manfaat yang lebih besar serta dapat digunakan sebagai dasar pengembangan sistem pendataan mahasiswa yang lebih kompleks di masa mendatang.